Testing Report

Test strategy, test cases, results, performance measurements and limitations

Project title	IntelliMetrics AI: A Governed Semantic Layer with TARA, an AI Assistant for Trusted Business Metrics
Document	Testing Report
Team members	Devi Priya, Priyanka, Akshay and Bhaskar
Guide	Dr. Asesh Kumar Tripathy
Institution	KL University
Academic year	2026-27
Date	24-Oct-2026 (IST)
Status	University project: academic capstone with synthetic demonstration data

Table of contents

- 1 Introduction
- 2 Test strategy
- 3 Test environment
- 4 Results by suite
- 5 Coverage
- 6 Functional test cases
- 7 Security testing
- 8 Performance measurements
  - 8.1 Dataset ingestion
  - 8.2 Metric query latency
  - 8.3 Dashboard load
  - 8.4 Ask TARA
  - 8.5 Conflict detection
- 9 Evaluation results (the project's research questions)
  - 9.1 Labelled ground truth for conflict detection
  - 9.2 Honest reading of these results
- 10 Defects found and fixed during testing
- 11 Verification of external services
- 12 Visual and accessibility checks
- 13 Test data
- 14 Reproducing the tests
- 15 Limitations: what was not tested

1 Introduction

This report describes how IntelliMetrics AI was tested and what the tests showed. Every figure in it is **substituted automatically from the files produced by the test runs** (JUnit XML, coverage JSON, Playwright JSON, and the benchmark script's JSON output stored in `docs/data`). None of the numbers was typed in by hand, so the report cannot disagree with the evidence. The final section states plainly what was **not** tested.

Headline	Result
Automated tests executed	325
Passed / failed	325 / 0
Back-end tests (SQLite)	136 run, 136 passed
Same back-end suite on real PostgreSQL 18	136 run, 136 passed
Front-end unit/component tests (Vitest)	40 run, 40 passed
Browser end-to-end tests (Playwright, Chromium)	13 run, 13 passed
Back-end statement coverage	82.8 % of 5,081 statements
Test machine	Windows-10-10.0.26300-SP0, Python 3.11.9, 22 logical CPUs

2 Test strategy

Testing follows the usual pyramid, with extra emphasis on the places where a wrong result would damage trust: the formula compiler (security), the planner (correctness of AI-assisted answers), conflict detection (the central feature) and workspace isolation (privacy).

Level	Tooling	What it proves
Unit	pytest	Formula lexer/parser/compiler, formatters, fiscal-calendar logic, planner rules, severity scoring, canonical forms
Integration / API	pytest with FastAPI `TestClient`	Every router, with a real database, real DuckDB queries over real Parquet files, background ingestion, SSE streaming
Security	pytest	Injection, token tampering, CSRF, rate limiting, secret leakage, path traversal, mass assignment, role escalation
Database portability	pytest against PostgreSQL 18	The same tests pass on the production-grade database, using schema built by Alembic migrations
Component	Vitest + Testing Library	Accessible UI primitives, route guard, formatters, API client (refresh, CSRF, SSE)
End-to-end	Playwright (Chromium)	Real browser against real front end and real back end: login, upload, metric creation, conflict workflow, Formula Lab, Ask TARA, dashboards, registration
Performance	`scripts/benchmark.py`	Ingestion throughput, query and dashboard latency, Ask TARA latency, conflict-detection time
Evaluation	in-app Evaluation module	Precision/recall of conflict detection on 27 labelled cases
Data quality	pytest on the seeded dataset	Indian formats, GST reconciliation, festival seasonality
Visual	Playwright screenshot sweep	Every page in light and dark mode, desktop and mobile, with console-error, 5xx and overflow checks

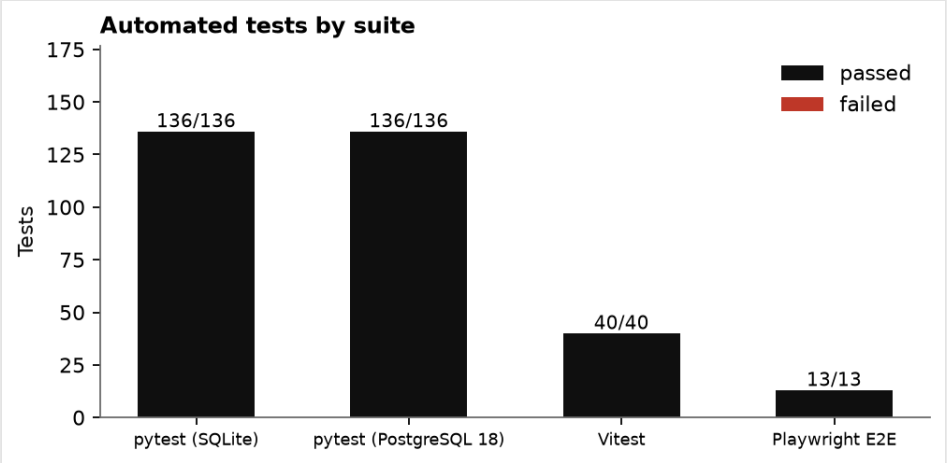
Determinism. The default AI mode is an offline rule-based planner, so every test is repeatable without network access or API keys. The AI-provider code paths (timeouts, malformed JSON, hallucinated columns) are tested with a mocked provider; no real OpenAI or Gemini call was made (see Limitations).

3 Test environment

Item	Value
Operating system	Windows-10-10.0.26300-SP0
Python	3.11.9 (back end), Node.js 22 (front end)
Databases used for tests	SQLite (default) and PostgreSQL 18.2 started as an isolated throw-away server on its own port
Analytics engine	DuckDB 1.1.3, Polars 1.18.0
Browser	Playwright Chromium
Dataset	`sales_transactions`: 62,000 rows, 24 columns, Parquet 1,743,248 bytes
Seeding time	2.7 s for the complete demo (users, 62,000 sales rows, metrics, dashboards)

4 Results by suite

Suite	Tests	Passed	Failed	Duration
pytest, SQLite (API, services, security)	136	136	0	14.6 s
pytest, real PostgreSQL 18.2 (same suite, Alembic-built schema)	136	136	0	12.8 s
Vitest + Testing Library (components, lib)	40	40	0	1.4 s
Playwright E2E (Chromium)	13	13	0	48.4 s
Total	325	325	0	



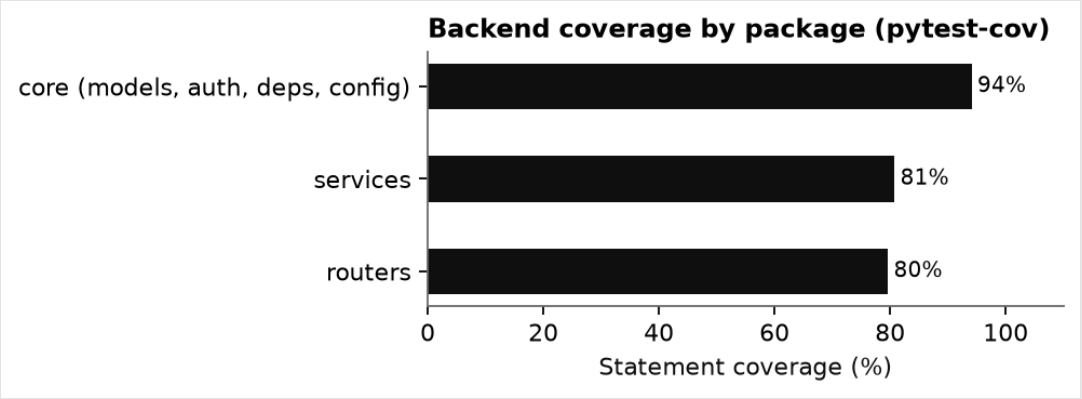
Automated tests by suite

5 Coverage

Backend (pytest-cov): **82.8%** of 5,081 statements. Frontend unit coverage (Vitest v8) over `src/lib`, `src/components` and `src/layouts`: statements **23.4%**, branches 70%, functions 52.1%; the 25 page components are exercised by the Playwright suite rather than by unit tests.

File (services and routers)	Statements	Missed	Coverage
`app/services/ai.py`	67	45	33%
`app/routers/users.py`	74	33	55%
`app/routers/notifications.py`	42	15	64%
`app/services/ingest.py`	260	86	67%
`app/services/conflicts.py`	203	61	70%
`app/routers/metrics.py`	184	52	72%
`app/routers/evaluation.py`	39	11	72%
`app/routers/dashboards.py`	132	37	72%
`app/routers/workspaces.py`	73	20	73%
`app/services/jobs.py`	11	3	73%
`app/services/metric_query.py`	71	19	73%
`app/services/formula.py`	700	166	76%
`app/routers/ai.py`	65	14	78%
`app/routers/datasets.py`	209	44	79%
`app/services/metrics.py`	183	29	84%
`app/routers/conflicts.py`	148	22	85%
`app/services/engine.py`	250	34	86%
`app/services/formatting.py`	54	7	87%
`app/routers/auth.py`	161	19	88%
`app/routers/reports.py`	34	4	88%
`app/services/planner.py`	402	44	89%
`app/services/tara.py`	158	16	90%
`app/routers/lab.py`	73	7	90%
`app/routers/chat.py`	168	14	92%
`app/services/dashboards.py`	52	4	92%
`app/routers/overview.py`	46	3	93%
`app/services/reports.py`	98	4	96%
`app/services/calendar_in.py`	61	2	97%
`app/services/evaluation.py`	122	3	98%

File (services and routers)	Statements	Missed	Coverage
`app/services/audit.py`	12	0	100%
`app/services/diffing.py`	14	0	100%



Back-end statement coverage for the lowest-covered service and router files

Coverage is reported honestly: files with lower coverage are mostly code paths that need external systems (live AI providers, external MySQL/PostgreSQL, Celery with Redis). Those paths were verified separately by scripts (section 11) rather than by unit tests, and the lines they execute are therefore counted as *missed* here.

6 Functional test cases

Each row below is linked to automated test results by a pattern match on the test name. The *Actual result* column shows how many underlying test cases matched and passed; a row is marked **FAIL** or **Not run** if no matching passing result exists, so the table cannot claim a pass that did not happen.

ID	Module	Test case	Input / steps	Expected result	Actual result	Status
TC-AUTH-01	Auth	Wrong password is rejected with the standard error envelope	POST /auth/login with a wrong password	401, code invalid_credentials, request_id present	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass
TC-AUTH-02	Auth	Protected endpoints require authentication	GET /metrics, /datasets, /overview … without cookie	401 on every protected route	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass
TC-AUTH-03	Auth	Cookies are httpOnly; refresh tokens rotate; logout revokes	login, refresh twice with the old token, logout	httpOnly flags; old refresh token 401; /me 401 after logout	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass
TC-AUTH-04	Auth	CSRF header required for cookie-authenticated mutations	POST /conflicts/detect without X-Requested-With	403 code csrf	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass
TC-AUTH-05	Auth	Registration creates an isolated workspace; duplicate and weak passwords rejected	register new user; register again; register '12345678'	201 admin of an empty workspace; 409; 400	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass
TC-AUTH-06	Auth	Forgot / reset password is token based and single use	forgot -> reset with link -> reuse the token	dev link returned only in development; second use 400; unknown email gives the same 200	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass

ID	Module	Test case	Input / steps	Expected result	Actual result	Status
TC-RBAC-01	RBAC	Role x endpoint permission matrix	16 role/endpoint combinations (business, analyst, reviewer, admin)	403 for roles lacking the capability, 200 otherwise	16/16 cases passed as expected; also 16/16 on PostgreSQL 18	Pass
TC-RBAC-02	RBAC	Reviewer cannot write; analyst cannot approve; reject needs a comment	PATCH as reviewer; approve as analyst; reject without comment	403, 403, 400; with comment status returns to draft	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass
TC-ISO-01	Isolation	A user in workspace B can never read workspace A data	GET/POST on A's dataset, metric, versions, execution, delete from B	404 everywhere; lists contain only own data; TARA cannot see A's metrics	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass
TC-ISO-02	Isolation	Admins cannot demote themselves or touch another workspace's users	PATCH self role; PATCH foreign user	403; 404	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass
TC-FOR-01	Formula	Formula compiles to parameterised SQL; division is zero-safe	SUM(...) - SUM(...) WHERE region = 'EU' ..., SUMIF, COUNTIF, ROUND	literals only as bound ? parameters; NULLIF(x, 0) for division	2/2 cases passed as expected; also 2/2 on PostgreSQL 18	Pass
TC-FOR-02	Formula	25 injection and invalid inputs are rejected	'; DROP TABLE', comments, read_csv(), UNION, backticks, nested aggregates, unknown columns ...	FormulaError with a clear message; nothing executed	25/25 cases passed as expected; also 25/25 on PostgreSQL 18	Pass
TC-FOR-03	Formula	String literals are never inlined into SQL	region = 'x'' OR ''1''='1'	value travels as a parameter; SQL text contains no payload	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass
TC-FOR-04	Formula	Depth and length limits	100 nested parentheses; 2000+ characters	FormulaError (too deep / too long)	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass

ID	Module	Test case	Input / steps	Expected result	Actual result	Status
TC-FOR-05	Formula	Metric references inline with their filters; cycles detected	[EuRev] / COUNT(*); [A] <-> [B]	FILTER (WHERE …) clause with parameter; Circular metric reference error	2/2 cases passed as expected; also 2/2 on PostgreSQL 18	Pass
TC-FOR-06	Formula	Dimension, grain, filter and limit builders reject hostile input	column 'region; DROP', grain "month'); –", operator '=; DROP'	FormulaError; row limit clamped to 10,000	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass
TC-FOR-07	Formula	Execution endpoint rejects injection and records failures	POST /execution/run with SELECT * FROM data and DROP TABLE	422 invalid_formula; failed runs appear in history	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass
TC-FOR-08	Formula	Canonical form ignores case and operand order	sum(Net_Sales)+SUM(x) vs SUM(x)+SUM(net_sales)	equal canonical strings; a/b differs from b/a	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass
TC-CON-01	Conflicts	Deterministic differences are classified	pairs differing in formula, aggregation, filter, unit, window, dataset	exactly the expected type per pair; no false positives for case/whitespace	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass
TC-CON-02	Conflicts	Precision and recall on labelled ground truth	27 labelled cases	automated precision = recall = 1.0; naive text diff precision < 0.8	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass
TC-CON-03	Conflicts	Detection is idempotent; TARA suggestions are labelled/skipped without AI	run detect twice	created = 0 the second time; tara.enabled false with a note	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass
TC-CON-04	Conflicts	Headline scenario: Finance Revenue vs official Revenue end to end	detect, assign, review, approve, resolve (rename), re-detect	ordered workflow enforced; new version created; audit trail complete; GST variant still open	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass

ID	Module	Test case	Input / steps	Expected result	Actual result	Status
TC-CON-05	Conflicts	GST-inclusive Revenue (Marketing) conflicts with ex-GST Revenue	detect	formula conflict on SUM(net_amount) vs SUM(net_sales)	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass
TC-VER-01	Versioning	Version rows are immutable	UPDATE / DELETE a metric_versions row through the ORM	ValueError: rows are immutable	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass
TC-VER-02	Versioning	Edits create versions; rollback creates a NEW version; history unchanged	edit, publish, edit, rollback, diff	versions 4,3,2,1 retained; v4 rollback_of 1; diff lists the formula change	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass
TC-VER-03	Versioning	Exactly one published definition per governed metric	publish a second 'Revenue'	409 already the published definition	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass
TC-VER-04	Versioning	Invalid formulas cannot be submitted	submit SUM(nope_col)	422 invalid_formula	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass
TC-ING-01	Ingestion	CSV upload is processed in the background and profiled	upload 2,001-row CSV with spaces in headers	202; status polling to ready; snake_case schema; date column detected; nulls and duplicates profiled	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass
TC-ING-02	Ingestion	Invalid uploads are rejected	.exe, empty file, fake parquet, binary csv, business user	400 / 403 before any processing	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass
TC-ING-03	Ingestion	A failing file ends in 'failed' with an error report	header-only CSV	dataset failed; job error_report.fatal set	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass

ID	Module	Test case	Input / steps	Expected result	Actual result	Status
TC-ING-04	Ingestion	JSON and Parquet formats	upload .json and .parquet	rows detected correctly	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass
TC-ING-05	Ingestion	Connection credentials are encrypted and never returned	create a connection; read the DB column and the API	ciphertext in DB; API shows ********; import with a malicious table name 400	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass
TC-AI-01	TARA	Offline generator is labelled and grounded in the schema	'Total quantity of units sold in the South region'	mode offline label; formula uses real columns; nothing auto-saved; unanswerable prompt reports an error	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass
TC-AI-02	TARA	AI timeout, malformed JSON, hallucinated column and repair	mocked provider	fallback labelled; one repair round; invalid output can be saved as draft but not submitted	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass
TC-AI-03	TARA	Forged or hallucinated query plans are rejected	mocked planner returning dimension 'ssn' or metric 'payroll_secret'	unsupported with an explanation; no query run	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass
TC-AI-04	TARA	Conversation history, SSE streaming and privacy	ask twice; stream; read as another user	4 messages with payload; status/result/text/done events; 404 for other users	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass
TC-IN-01	Indian context	Lakh/crore digit grouping	1245300, 123456789 …	12,45,300 and 12,34,56,789	7/7 cases passed as expected; also 7/7 on PostgreSQL 18	Pass
TC-IN-02	Indian context	INR formatting and compact forms	inr(1245300), compact(1.2 Cr, 45 L)	₹12,45,300; ₹1.2 Cr; ₹45 L	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass

ID	Module	Test case	Input / steps	Expected result	Actual result	Status
TC-IN-03	Indian context	April-March fiscal quarters	fq_bounds for Q1-Q4; fq_of dates around 31-Mar/01-Apr	Q1 Apr-Jun … Q4 Jan-Mar	2/2 cases passed as expected; also 2/2 on PostgreSQL 18	Pass
TC-IN-04	Indian context	14 natural-language time expressions	last quarter, this FY, FY 2025-26, FY26, Q3 FY26, first quarter of last financial year …	exact ISO start/end dates on the fiscal calendar	14/14 cases passed as expected; also 14/14 on PostgreSQL 18	Pass
TC-IN-05	Indian context	Fallbacks and previous-period logic	last quarter of this FY when no quarter has completed; previous quarter of Diwali	Q4 of previous FY; Q2 FY 2025-26	2/2 cases passed as expected; also 2/2 on PostgreSQL 18	Pass
TC-IN-06	Indian context	Dates as DD-MMM-YYYY in IST	fmt_date, UTC to IST conversion	05-Mar-2026; 20:00 UTC -> 01-Jan-2026 01:30 IST	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass
TC-IN-07	Indian context	Synthetic dataset is Indian, large and well formed	profile of sales_transactions; customers preview	62,000 rows; five regions, channels, payment methods; valid 6-digit PINs; +91 90000 numbers; example.com e-mails	2/2 cases passed as expected; also 2/2 on PostgreSQL 18	Pass
TC-IN-08	Indian context	GST amounts reconcile and slabs are official	net_amount - net_sales - gst_amount; distinct gst_rate	difference < ₹5; slabs within {0, 5, 12, 18, 28}	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass
TC-IN-09	Indian context	Festive seasonality and COD returns are present in the data	Diwali 2025 daily revenue vs previous quarter; return rate COD vs UPI	festival run-rate > 1.3x; COD return rate > 1.5x UPI	2/2 cases passed as expected; also 2/2 on PostgreSQL 18	Pass
TC-IN-10	Indian context	The five suggested Indian questions return real, data-backed answers	five questions via /chat/ask	kind answer, rows present, rupee symbol, no dollar amounts; API suggests them	8/8 cases passed as expected; also 8/8 on PostgreSQL 18	Pass

ID	Module	Test case	Input / steps	Expected result	Actual result	Status
TC-IN-11	Indian context	Diwali-season vs previous-quarter comparison	Compare Diwali-season sales with the previous quarter	two ranges labelled; per-day note for unequal lengths	2/2 cases passed as expected; also 2/2 on PostgreSQL 18	Pass
TC-IN-12	Indian context	Exports use INR and IST	dashboard CSV; metrics CSV	rupee symbol, lakh grouping, DD-MMM-YYYY HH:MM IST	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass
TC-DASH-01	Dashboards	All three templates render real data	create Operations template; load widget data	every widget ok; KPI has a value	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass
TC-REP-01	Reports	Evaluation run and exports in CSV, XLSX and PDF	POST /evaluation/run; export metrics, conflicts, evaluation, dashboard	precision/recall >= 0.9; files have correct magic bytes	1/1 case passed as expected; also 1/1 on PostgreSQL 18	Pass
TC-FE-01	Frontend	Theme toggle persists and applies the dark class	click Dark, Light, System	class + localStorage updated	1/1 case passed as expected	Pass
TC-FE-02	Frontend	Images: alt text, lazy loading, B/W filter, gradient fallback; all config entries valid	SafeImage load / error; images.ts audit	fallback rendered on error; every entry has URL, alt and source	3/3 cases passed as expected	Pass
TC-FE-03	Frontend	Tables, pager, diff, validation panel, route guard	DataTable sort/keyboard; Pager; FormulaDiff; RequireCap	behaviour and accessibility attributes as specified	10/10 cases passed as expected	Pass
TC-FE-04	Frontend	INR/lakh/crore and IST formatters	formatValue, compactIN, axisFormat, fmtDate/fmtDateTime/fmtDay	₹12,45,300; ₹1.2 Cr; 01-Jan-2026 01:30 IST	4/4 cases passed as expected	Pass
TC-FE-05	Frontend	API client: CSRF header, error envelope, 401 refresh-and-retry, SSE parser	mocked fetch	single refresh then retry; sign-out when refresh fails; stream events parsed	7/7 cases passed as expected	Pass
TC-FE-06	Frontend	UI primitives are accessible	forms, buttons, dialogs, tabs, toasts, progress, empty/error states	roles, labels, aria-live regions present	12/12 cases passed as expected	Pass

ID	Module	Test case	Input / steps	Expected result	Actual result	Status
TC-FE-07	Frontend	Landing page has no pricing and shows the University Project badge	render landing	no pricing/plan/billing text; academic CTAs; rupee figure	3/3 cases passed as expected	Pass
TC-E2E-01	E2E	Login with the demo panel, bad password, logout, route protection	browser	dashboard after login; login page after logout	1/1 case passed as expected	Pass
TC-E2E-02	E2E	All demo roles sign in and see role-appropriate navigation	4 roles	nav items shown/hidden per role	1/1 case passed as expected	Pass
TC-E2E-03	E2E	403 and 404 pages	business user opens /app/lab; unknown URL	Forbidden and Not found pages	1/1 case passed as expected	Pass
TC-E2E-04	E2E	Theme toggle light/dark/system with no flash	toggle, reload, init script	class and localStorage persist; pre-paint script applies theme	1/1 case passed as expected	Pass
TC-E2E-05	E2E	Upload a dataset and see profiling	upload 3,000-row CSV	ready; row count; profile and schema tabs	1/1 case passed as expected	Pass
TC-E2E-06	E2E	Create a metric with live validation and submit it	invalid then valid formula	validation messages; draft saved; in review	1/1 case passed as expected	Pass
TC-E2E-07	E2E	Detect conflict, assign, review, approve, resolve	Reviewer in the UI	Finance and Marketing Revenue conflicts; new version v2 for the renamed metric	1/1 case passed as expected	Pass
TC-E2E-08	E2E	Formula Lab executes and rejects injection	SUM(net_sales); injection attempt; group by region	rupee-grouped value; rows scanned; error 'Nothing was executed'	1/1 case passed as expected	Pass
TC-E2E-09	E2E	Ask TARA: Indian fiscal answers in INR with chart and sources	suggested questions, UPI vs COD, Diwali comparison, weather question	tables, line / grouped charts, per-day note, refusal for the weather question	1/1 case passed as expected	Pass
TC-E2E-10	E2E	Landing page has no pricing; University Project badge; About page	browser	no banned words; badge and footer note visible	1/1 case passed as expected	Pass
TC-E2E-11	E2E	Indian formatting on the dashboard and audit log	Sales Overview KPI; audit log dates	₹ with lakh/crore grouping; DD-MMM-YYYY HH:MM IST	1/1 case passed as expected	Pass

ID	Module	Test case	Input / steps	Expected result	Actual result	Status
TC-E2E-12	E2E	Build a dashboard from a template, edit layout, add widget, save, export menu	Operations template	widget count persists after reload; export options visible	1/1 case passed as expected	Pass
TC-E2E-13	E2E	Register a workspace and reset the password via the dev banner link	register, forgot, reset, login	dev banner shown; new password works	1/1 case passed as expected	Pass

7 Security testing

Threat	Test	Result
SQL injection via formulas	25 hostile formulas (`; DROP TABLE` , comments, `read_csv` , `UNION` , backticks, nested aggregates, unknown columns)	all rejected before execution; literals are bound parameters
SQL injection via query strings	`' OR 1=1 --` , `'; DROP TABLE metrics; --` , `UNION SELECT password_hash ...` on three list endpoints	inert; the tables remain intact
Forged AI plan	Mocked planner returns a forbidden dimension and an unknown metric	rejected by `validate_plan` ; nothing executed
Token forgery	wrong signature, expired token, refresh token used as access token, `alg=none`	every variant returns 401
CSRF	cookie-authenticated POST without the custom header	403
Brute force	repeated logins from one address	429 after the configured limit
Cross-tenant access	user of workspace B requests workspace A's dataset, metric, versions, execution	404 everywhere
Privilege escalation	Business User changes own role; Analyst creates an Admin	403 / role unchanged
Secret leakage	search API responses for password hashes, Fernet ciphertext, secret key, API-key names	none present; connection passwords shown only as `********`
Path traversal in uploads	file name `../../../evil name.csv`	stored inside the data directory under a generated name
Malicious files	`.exe` , empty file, fake Parquet, binary CSV	rejected before processing
Information disclosure in errors	malformed login, unknown ids	uniform error envelope with a request id; no stack traces

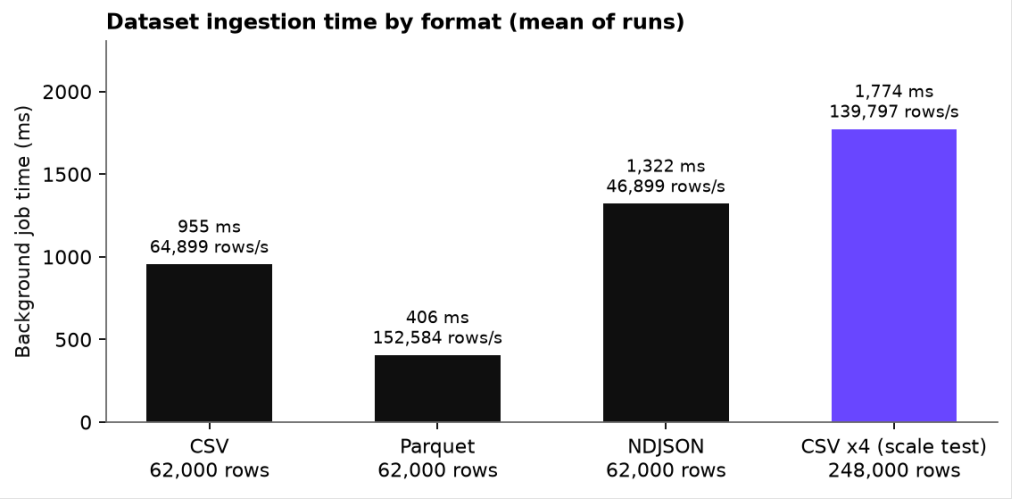
These checks are automated (`backend/tests/test_security.py` , `test_auth_rbac.py` , `test_formula.py` , `test_governance.py`). They are an application-level assessment; **no external penetration test or dependency-vulnerability scan was performed**.

8 Performance measurements

All values were measured on the machine described in section 3, on the 62,000-row dataset, with the offline AI mode, SQLite for application data and the in-process job runner. A production deployment on PostgreSQL with Celery will have different absolute numbers; the relative picture (DuckDB over Parquet is fast, ingestion is the heavy step) is what matters.

8.1 Dataset ingestion

Format	Rows	File (MB)	Runs	Job time mean (ms)	Job p95 (ms)	Upload to ready (ms)	Rows / second
CSV	62,000	11.8	3	955	999	1,065	64,899
Parquet	62,000	1.7	3	406	420	489	152,584
NDJSON	62,000	31.2	3	1,322	1,376	1,504	46,899
CSV x4 (scale test)	248,000	47.6	1	1,774	n/a (1 run)	2,063	139,797

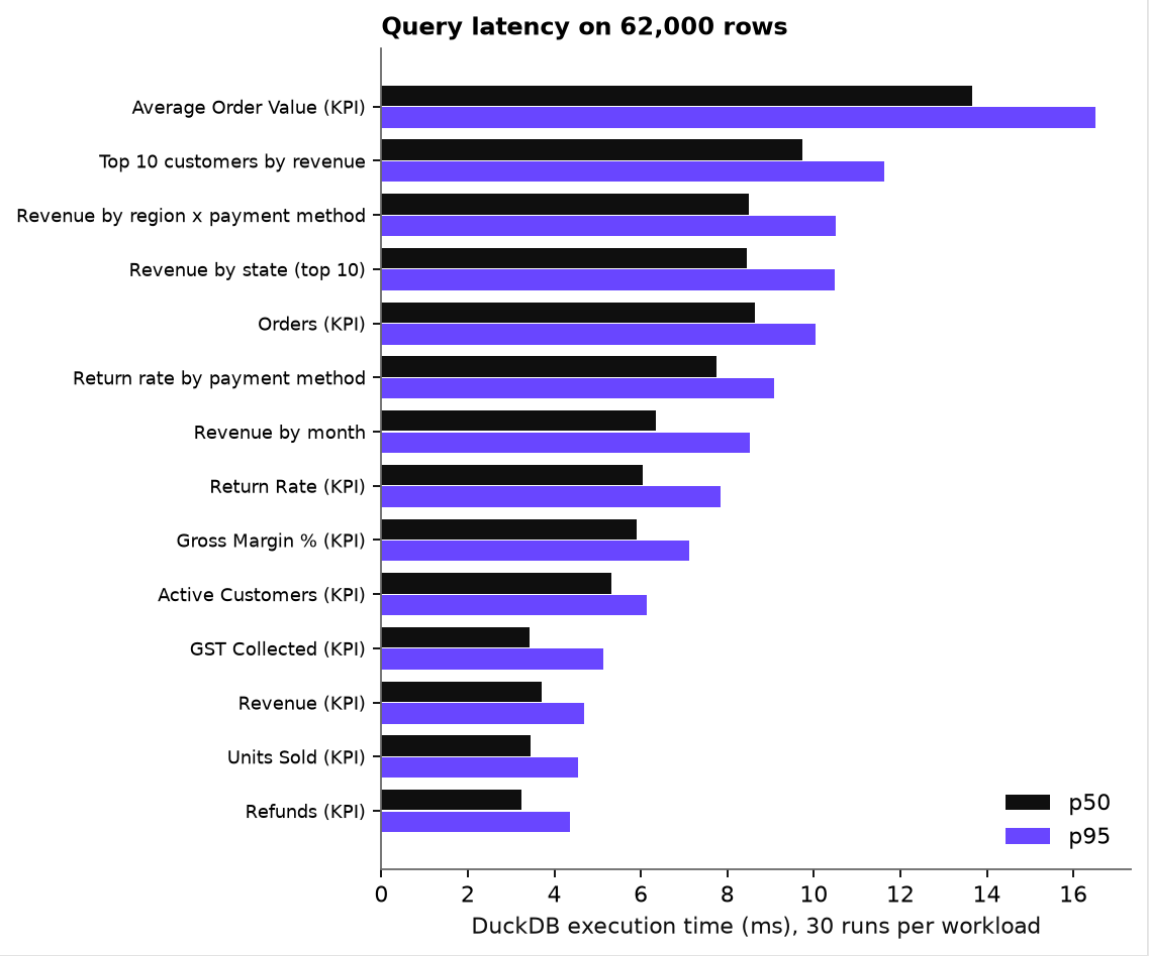


Dataset ingestion time by format

8.2 Metric query latency

Workload (30 runs each)	Engine p50 (ms)	Engine p95 (ms)	HTTP round-trip p50 (ms)	HTTP p95 (ms)
Return Rate (KPI)	6.04	7.85	31.91	36.89
Active Customers (KPI)	5.33	6.14	25.59	35.6
Gross Margin % (KPI)	5.91	7.11	23.67	27.48
GST Collected (KPI)	3.42	5.13	21.78	26.45
Refunds (KPI)	3.23	4.37	24.17	32.13
Average Order Value (KPI)	13.67	16.51	41.28	50.94

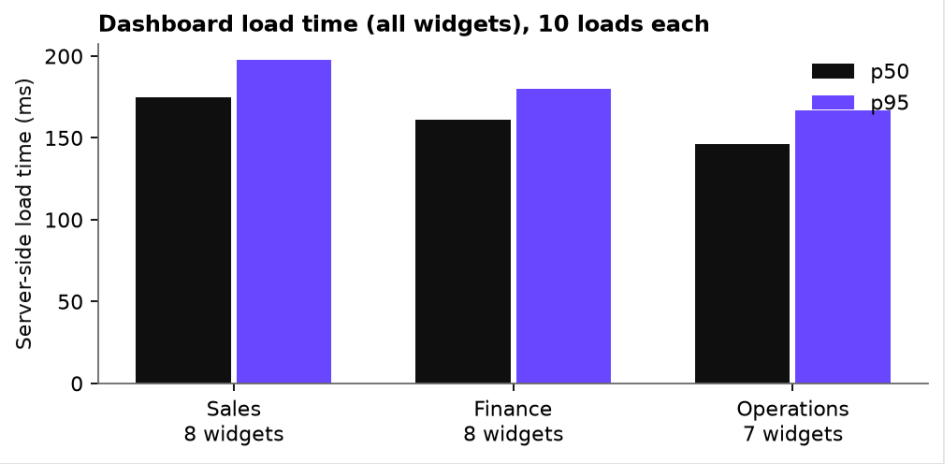
Workload (30 runs each)	Engine p50 (ms)	Engine p95 (ms)	HTTP round-trip p50 (ms)	HTTP p95 (ms)
Units Sold (KPI)	3.44	4.55	24.93	33.11
Orders (KPI)	8.64	10.03	29.45	32.08
Revenue (KPI)	3.71	4.68	24.28	28.27
Revenue by month	6.35	8.51	27.32	35.87
Revenue by state (top 10)	8.44	10.49	32.1	38.19
Revenue by region x payment method	8.5	10.5	30.33	33.91
Top 10 customers by revenue	9.73	11.62	30.27	33.49
Return rate by payment method	7.75	9.09	27.98	30.39
All 14 workloads (420 samples)	**6.37**	**12.77**	**27.85**	**40.17**



Engine and HTTP latency per workload

8.3 Dashboard load

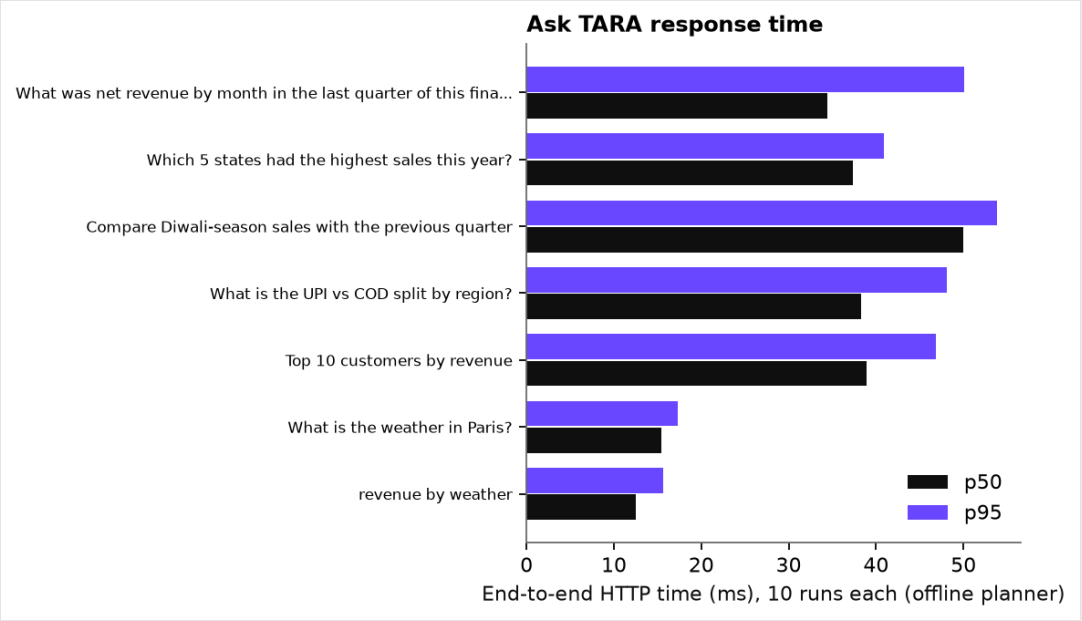
Dashboard	Widgets	Server load p50 (ms)	p95 (ms)	HTTP p50 (ms)	Failed widgets (all loads)
Sales	8	174.95	197.51	186.23	0
Finance	8	161.05	179.76	170.62	0
Operations	7	146.1	166.81	156.05	0



Dashboard load time

8.4 Ask TARA

Question (10 runs each, offline planner)	Outcome	p50 (ms)	p95 (ms)
What was net revenue by month in the last quarter of this financial year?	answer	34.4	50.09
Which 5 states had the highest sales this year?	answer	37.36	40.91
Compare Diwali-season sales with the previous quarter	answer	50.01	53.84
What is the UPI vs COD split by region?	answer	38.28	48.05
Top 10 customers by revenue	answer	38.92	46.88
What is the weather in Paris?	unsupported	15.47	17.26
revenue by weather	unsupported	12.5	15.66



Ask TARA latency by question

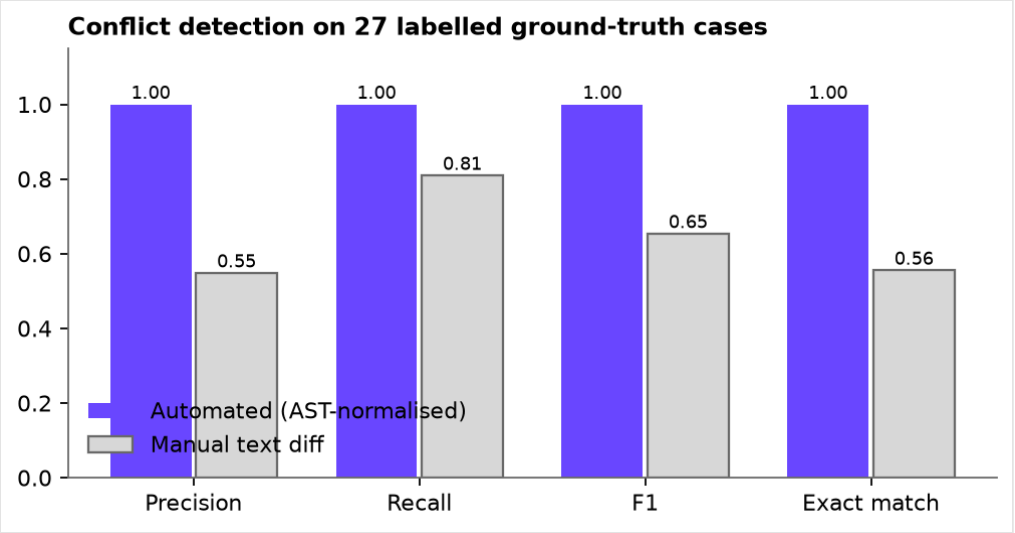
The latency of Ask TARA here is that of the **offline** planner. A live AI provider adds one network round-trip for planning (typically a few hundred milliseconds to several seconds); that was not measured.

8.5 Conflict detection

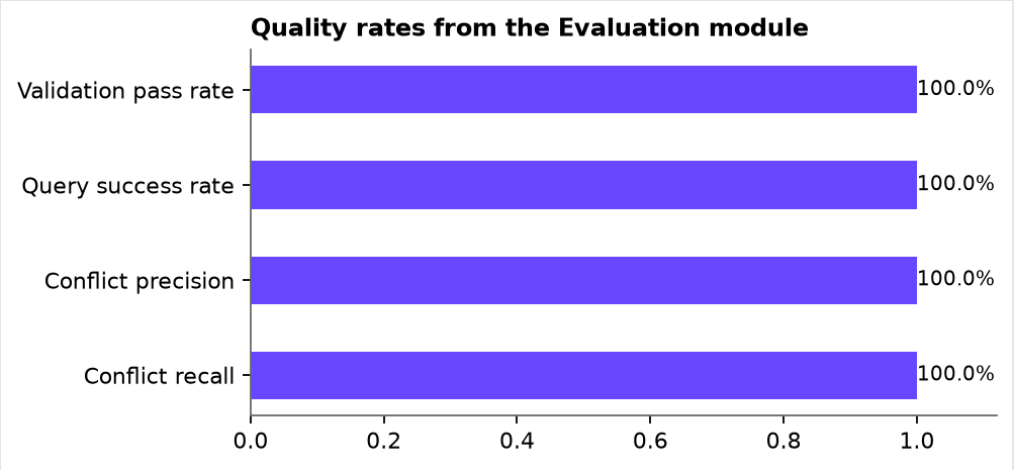
Detection over the seeded workspace created 4 conflicts and took a mean of 15.2 ms (p95 22.1 ms over 5 runs).

9 Evaluation results (the project's research questions)

Measure	Result	Detail
Dataset processing (avg job time)	653 ms	16 successful jobs; 86,494 rows/s
Query latency (benchmark of every live metric)	avg 4.6 ms, p95 8.92 ms	9 metrics, full 62,000-row dataset
Validation pass rate	100%	23/23 metric definitions re-validated
Query success rate	100.0%	719/719 executions in the run history
Conflict detection precision / recall (automated)	1.00 / 1.00	27 labelled cases; TP 21, FP 0, FN 0
Conflict detection precision / recall (manual text diff baseline)	0.55 / 0.81	TP 17, FP 14, FN 4
AI suggestion acceptance rate (scripted offline session)	80%	8 accepted of 10 generations; 100% of outputs valid
Dashboard load time (avg of 5 stored dashboards)	167.2 ms	p95 230.2 ms; 5 dashboards
Metric creation: manual vs AI-assisted	360 s (assumed) vs 60.0 s	TARA generation measured at 0.0 s on average + 60 s assumed human review; estimated saving 83.3%
Conflict detection: manual vs automated	225 s (assumed) vs 0.48 ms	5 candidate pairs; manual review assumed at 45 s per pair



Conflict-detection quality: automated detector versus a naive text comparison



Evaluation rates

9.1 Labelled ground truth for conflict detection

The detector was tested against 27 cases with known correct answers, including identical definitions that differ only in case or spacing (which must *not* be flagged), single differences of each type, combinations, and the GST-inclusive versus ex-GST revenue case.

Case	Scenario	Expected	Automated	Correct	Manual text diff	Correct
c01	Identical definitions	none	none	yes	none	yes
c02	Case and whitespace only	none	none	yes	formula	NO
c03	Commutative addition reordered	none	none	yes	formula	NO
c04	Finance subtracts refunds	formula	formula	yes	formula	yes
c05	COUNT vs COUNT_DISTINCT	aggregation	aggregation	yes	formula	NO
c06	SUM vs AVG	aggregation	aggregation	yes	formula	NO
c07	Region filter only on one side	filter	filter	yes	filter	yes
c08	Filter conjunct order swapped	none	none	yes	filter	NO
c09	Currency differs INR vs USD	unit	unit	yes	unit	yes
c10	INR vs percent	unit	unit	yes	unit	yes
c11	all_time vs ytd	date_range	date_range	yes	date_range	yes
c12	Same formula different dataset	dataset	dataset	yes	dataset	yes
c13	Different measure and filter	filter, formula	filter, formula	yes	filter, formula	yes

Case	Scenario	Expected	Automated	Correct	Manual text diff	Correct
c14	Unit missing on one side	unit	unit	yes	unit	yes
c15	Aggregation, unit and window all differ	aggregation, date_range, unit	aggregation, date_range, unit	yes	date_range, formula, unit	NO
c16	Same window, different date column	date_range	date_range	yes	date_range	yes
c17	Ratio inverted	formula	formula	yes	formula	yes
c18	Filter in WHERE clause vs filters field	none	none	yes	filter, formula	NO
c19	IN list order swapped	none	none	yes	filter	NO
c20	Revenue vs net-of-discount	formula	formula	yes	formula	yes
c21	Distinct customers vs rows	aggregation	aggregation	yes	formula	NO
c22	Identical ratio formulas	none	none	yes	formula	NO
c23	Revenue excluding GST vs including GST (invoice value)	formula	formula	yes	formula	yes
c24	GST added to net sales, operands reordered	none	none	yes	formula	NO
c25	Finance nets refunds off Indian revenue	formula	formula	yes	formula	yes
c26	Financial-year-to-date vs all time	date_range	date_range	yes	date_range	yes
c27	UPI-only filter written two ways	none	none	yes	filter, formula	NO

9.2 Honest reading of these results

- The perfect precision and recall of the automated detector hold for these labelled cases, **which were written by the author of the detector**. They show that the rules behave as designed; they are not an independent benchmark and should not be read as a general accuracy claim.
- The *manual text-diff* baseline is a simple string comparison used as a stand-in for a person eyeballing two formulas. It is not a human study.

- The **time-saving figures for metric creation and conflict review depend on assumed manual durations** (360 s per metric and 45 s per pair). They are labelled as assumptions in the application and here. Only the machine times (milliseconds) are measured.
- The AI acceptance rate is from a **scripted offline session** of 10 prompts, two of them deliberately unanswerable; it is not a user study.

10 Defects found and fixed during testing

#	Defect found	How it was found	Fix
1	Opening the dataset **Profiling** tab crashed the whole page (blank screen): `DataTable` treated `undefined === undefined` as "sorted column" and called `.startsWith` on an undefined sort key.	Playwright upload test (screenshot of a white page) and a console capture.	Guard `active = !!sort && !!c.sortKey && ...`; a Vitest regression test now renders the table without a sort prop.
2	TARA planner read "this year" as a yearly grouping and "weekly" as monthly because a shared regular-expression alternation matched the bare word.	Manual probe of 7 questions after the first end-to-end run.	Replaced with explicit grain detection (`by/per/each <unit>` , `monthly/weekly/...`); covered by pytest.
3	"Top 5 categories by revenue": the phrase after *by* (revenue) was taken as the dimension, so TARA answered "cannot break down by revenue".	Same probe.	Dimension extraction now prefers the noun after *top N* / *which N*, and ignores a *by* phrase that only names the metric.
4	A synonym table mapped "customers" to the *Active Customers* metric, which also stopped "Top 10 customers by revenue" from finding the customer dimension.	Probe of the Indian suggested questions.	Removed the over-eager synonyms; metric tokens are no longer used to hide dimension words.
5	"Compare Diwali-season sales with the previous quarter" compared against the *wrong* quarter (the generic "last quarter" rule fired on the words "previous quarter").	Probe of the Indian suggested questions (Q2 FY 2026-27 shown instead of Q2 FY 2025-26).	Relative phrases after "compare … with" now call `previous_period()` on the first range; asserted by `test_diwali_comparison_is_data_backed_and_labelled` .
6	UPI-vs-COD answer text reported the **share percentage** formatted as rupees (the share column was appended before the text was composed).	Probe output.	Compose the text from the value rows first, then append the share column; asserted in `test_top_states_customers_and_split` .

#	Defect found	How it was found	Fix
7	The password-reset link printed in development pointed at a different origin (`FRONTEND_URL`) than the app under test, so the token was rejected.	Playwright register/reset test.	E2E backend now sets `FRONTEND_URL` ; the dev banner renders an in-app (path-only) link regardless of origin.
8	Alembic autogenerate rendered the partial unique index predicate as `is_live IS 1` , which is invalid PostgreSQL.	Reviewing the generated migration before running it.	Predicate written as `IS TRUE` in the model and migration; later confirmed on real PostgreSQL 18 (`CREATE UNIQUE INDEX ... WHERE (is_live IS TRUE)`).
9	Vite dev server exited because `jspdf` references the optional dependency `html2canvas` .	First attempt to start the frontend in the browser pane.	`optimizeDeps.exclude: ["jspdf"]` , jsPDF loaded lazily, optional modules marked external in the build.
10	The API client cached a resolved token-refresh promise, so a later 401 could skip the refresh and never sign the user out.	New Vitest test “signs the user out when the refresh fails”.	Clear the cached promise in `finally` ; test passes.
11	Dashboard KPI “Validation pass rate” showed `—%` and counted only Formula Lab checks (zero on a fresh workspace).	Visual review of the screenshots.	The KPI re-validates every live metric definition; the suffix is hidden when there is no value.
12	A React hook (`useMemo`) was called after an early return on the Versions page (rules-of-hooks violation).	Code review while writing the page.	Removed the memo before the first conditional return.
13	Strings generated by TARA or users could exceed PostgreSQL column lengths (SQLite does not enforce them).	Static review for PostgreSQL strictness before the PostgreSQL run.	Server-side length checks for metric fields; notifications truncated.
14	`time_window = "ytd"` meant *calendar* year to date, which is wrong for an Indian financial year.	Indian-context rework.	`ytd` now means financial-year-to-date (1 April); tested by `test_metric_window_ytd_is_financial_year_to_date` .

#	Defect found	How it was found	Fix
15	pytest fixture ordering assumption: a test asked for `status=all`, a value the API intentionally does not implement (the UI omits the parameter).	Running the new Indian-context tests in isolation.	Test corrected; behaviour unchanged.
16	On a phone-width screen the dashboard overflowed horizontally (page scroll width 660 px on a 390 px screen) once a long metric name appeared under "Recent metric changes": a one-column CSS grid sized itself to the widest unbreakable text.	Final screenshot sweep (horizontal-overflow check, mobile dark dashboard).	The grids now declare `grid-cols-1`, so columns can shrink and long text truncates; the sweep then reported no problems in light and dark, desktop and mobile.

11 Verification of external services

Because Docker was not available on the development machine, the services that Docker Compose would provide were verified **individually, as isolated throw-away servers on spare ports**, without touching any service the user already had running.

Service	What was run	Result
PostgreSQL 18.2	Alembic migrations from an empty database, then the complete pytest suite (136 tests)	passed
PostgreSQL 18.2 as an import source	Create a connection, store the password encrypted, list tables, stream a table into Parquet, attempt a write	30,000 rows imported in 335 ms; password encrypted at rest; a write was rejected by the read-only session
Redis + Celery	Real Celery worker (`--pool=solo`) consuming from a real Redis broker; a 62,000-row CSV upload processed by the worker, application data in PostgreSQL	status `ready` , runner `celery` , job 748 ms, upload to ready 2,095 ms; progress states pending, processing, ready all observed
MySQL 8.0.40 as an import source	The same connection test against MySQL	30,000 rows imported in 299 ms; password encrypted at rest; a write was rejected by the read-only session
Demonstration walkthrough	`scripts/verify_walkthrough.py` against the running back end after a full reset and re-seed: six logins, role restrictions, conflict workflow, Formula Lab with an injection attempt, eight Ask TARA questions, dashboard widgets, audit log and cross-workspace isolation	36 of 36 checks passed
Screenshot sweep	`frontend/e2e/shots.mjs` : every page in light and dark, desktop and 390 px phone, checking console errors, HTTP 5xx and horizontal overflow	The sweep found one real overflow bug (defect 16), fixed; the final run reported no problems
Image URLs	HTTP request for each of the 37 stock-image URLs used by the site	37 returned HTTP 200
Commercial-language check	`scripts/check_no_pricing.py` over all source, configuration and documentation files	0 unexplained hits; 205 allow-listed hits that use an unrelated sense of a word (a TARA “plan”, the Alembic `upgrade` command, a product unit price in the sales data, or a test that asserts pricing is absent)

12 Visual and accessibility checks

A Playwright sweep (`frontend/e2e/shots.mjs`) visited every page in light and dark mode at desktop and phone widths and checked for console errors, HTTP 5xx responses and horizontal overflow. The pages are shown in the User Guide and listed in `docs/screenshots` . Accessibility is supported by semantic HTML, labelled form controls, visible focus rings, `aria-live` regions for toasts, keyboard-operable tables and tabs, and a Vitest suite that asserts these attributes. **No formal WCAG audit or screen-reader test was performed.**

13 Test data

The demonstration data is synthetic and generated by `backend/app/seed_data.py` with a fixed random seed, so every run produces the same rows. It models an Indian omnichannel retailer: 62,000 order lines across 27 cities in 18 states across five regions, GST slabs of 0/5/12/18/28 %, payment methods (UPI, cards, net banking, wallets, cash on delivery), festive-season demand peaks (Diwali, Dussehra, Pongal/Sankranti, Eid, Holi, End-of-Season sale) and a higher return rate for cash on delivery. Customer names, phone numbers (`+91 90000 ...`) and e-mails (`example.com`) are fictional. No real person's data is used anywhere.

14 Reproducing the tests

```
# back end (SQLite)
cd backend && python -m pytest tests -q --cov=app

# back end against PostgreSQL (after creating an empty database and running alembic upgrade head)
TEST_DATABASE_URL=postgresql://user:pass@host:5432/db TEST_SKIP_CREATE_ALL=1 python -m pytest tests -q

# front end
cd frontend && npx vitest run --coverage.enabled
npx playwright test            # starts its own back end and front end on ports 8100 and 5273

# performance benchmark
python scripts/benchmark.py
```

15 Limitations: what was not tested

An honest list, so that nobody reads more into this report than it supports.

1. **Docker and Docker Compose were never run.** Docker is not installed on the development machine. The `Dockerfile`s, `docker-compose.yml` and Nginx configuration were written and reviewed, and every service they start was verified individually (PostgreSQL, Redis + Celery, MySQL), but the container images were **not built** and the Compose stack was **not started**.
2. **No live AI provider was used.** There was no OpenAI or Gemini key. The provider adapters were tested with mocked responses only; real-model behaviour, cost and latency are unmeasured. The offline deterministic mode is what was exercised end to end.
3. **No load or concurrency testing** beyond single-user latency measurements. Behaviour with many simultaneous users is unmeasured. The rate limiter is per-process, so it would need Redis-backed storage to be correct with several API workers.
4. **No external security assessment.** Only the application-level tests above.
5. **Only Chromium was used** for browser tests; Firefox and Safari were not tested. Layout was checked at desktop and 390 px phone widths only.
6. **No usability study with real users**; the manual-effort comparisons use stated assumptions.
7. **The Windows development machine only.** Linux and macOS were not used, although the code uses portable paths and the Compose files target Linux containers.
8. **External MySQL/PostgreSQL import** was verified against isolated local servers, not against a managed cloud database.
9. **Backups, restore and zero-downtime updates** are documented in the Deployment Guide but were not exercised.
10. **The GitHub Actions workflow** (`.github/workflows/ci.yml`) was written but never run on GitHub; the same commands were run by hand locally instead (the Docker build job in it was not run at all).